UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencias de la Computación

Desarrollo de Software - CC3S2A

Tarea-1:

Enrutamiento en una aplicación - Cliente más fiel por socio

Alumno:

- López Arteaga, César Omar - 20236743A

Profesor: Manuel Alejandro Quispe Torres

Contenido

1. Enrutamiento en una aplicación **3**

1.1. Objetivo del Programa 5

1.2. Estructura General del Programa 5

1.3. Lectura de Datos 8

1.4. Representacion de Rutas 8

1.5. Estructura de Datos 8

1.6. Procesamiento de Transiciones 9

1.7. Regla de Coincidencia 9

1.8. Captura de Parametros 9

1.9. Determinacion del Resultado 10

1.10. Estructuras de Datos Utilizadas 10

1.11. Conclusion 10

2. Cliente más fiel por socio **11**

2.1. Objetivo del Programa 13

2.2. Estructura General del Programa 13

2.3. Importación de Módulos 13

2.4. Lectura de Datos 14

2.5. Mapeo de Terminales 15

2.6. Estructura de Conteo 15

2.7. Procesamiento de Transacciones 16

2.8. Determinación del Cliente Más Fiel 17

2.9. Escritura de Resultados 18

2.10. Estructuras de Datos Utilizadas 18

2.11. Conclusión 18

Para el desarrollo del trabajo, se usara el el lenguaje python en el id de visual code. Se cargargara la soluciones de los codigo en el Github creado para el curso.

El link del respositorio de la tarea es: Código fuente de Tarea 1

1. Enrutamiento en una aplicación

Crea una función que simule el comportamiento de enrutamiento en una aplicación de una sola página. La función recibe como entrada un conjunto de rutas con su contenido correspondiente y una secuencia de transiciones que el usuario puede realizar. La función debe generar el contenido de cada ruta.

Requisitos

- Cada ruta tiene un `path` y contenido asociado. Algunas rutas pueden incluir parámetros.
- Para cada transición, la función debe determinar la ruta correspondiente y devolver el contenido asociado. Si no se encuentra ninguna ruta, debe devolver:

  `Error 404: No encontrado`

- Las rutas pueden contener parámetros precedidos por dos puntos, por ejemplo:

  `/user/:id`

  En este caso, se añade el valor del parámetro al contenido, seguido de un espacio. Por ejemplo:

  UserPage 42

  para la ruta `/user/:id` y la transición:

  /user/42

Formato de entrada

- La primera línea contiene un número entero N: el número de rutas.

- Las siguientes N líneas describen las rutas en el formato:

```
ruta contenido
```

- La siguiente línea contiene un número entero M: el número de transiciones.
- Las siguientes M líneas contienen la secuencia de transiciones para las que se debe determinar el contenido.

Formato de salida

Para cada transición, muestra la cadena de respuesta que coincida con la ruta. Si no se encuentra ninguna ruta, devuelve:

```
404 Not Found
```

Ejemplo de entrada

```
3
/ HomePage
/profile ProfilePage
/user/:id UserPage {id}
4
/
/profile
/user/42
/settings
```

Ejemplo de Salida

```
HomePage
ProfilePage
UserPage 42
404 Not Found
```

Resolucion

En esta parte se tendra un archivo de entrada de la data, denominada input.txt. La salida del programa sera por terminal, mostrando el contenido correspondiente a cada transicion solicitada y un archivo de salida de resultado denominado output.txt.

1.1. Objetivo del Programa

El objetivo del programa es simular el comportamiento de enrutamiento de una aplicacion de una sola pagina. Para cada transicion realizada por el usuario, el programa determina la ruta correspondiente y muestra su contenido asociado.
En caso de que la ruta no exista, se devuelve el mensaje:

404 Not Found

Si la ruta contiene parametros (por ejemplo, /user/:id), el valor del parametro se añade al contenido.

1.2. Estructura General del Programa

El programa se organiza en las siguientes etapas:

- Lectura de rutas
- Construccion de estructuras de datos
- Procesamiento de transiciones
- Comparacion de rutas
- Generacion de resultados

Asi el programa general tendra de funciones los cuales son:

- `dividir_ruta(ruta)` : este convierte una ruta tipo string en una lista de segmentos. Por ejemplo:

$$\texttt{/user/:id} \rightarrow [\texttt{user},\ \texttt{:id}]$$

$$\texttt{/profile} \rightarrow [\texttt{profile}]$$

$$\texttt{/} \rightarrow [\ ]$$

```python
#convierte una ruta tipo string en una lista de segmentos
def dividir_ruta(ruta):
    # caso especial: ruta raiz "/"
    if ruta == "/":
        return []

    # elimina "/" al inicio y final, luego separa por "/"
    return ruta.strip("/").split("/")
```

- `coincide_ruta(ruta_definida,transicion)` : Esta funcion determina si una transicion coincide con una ruta definida. Asi los parametros de la funcion son: `ruta_definida`, es la ruta registrada (ejemplo `/user/:id`) y la transicion es la ruta ingresada (ejemplo `/user/42`).

```python
#determina si una transicion coincide con una ruta definida.
def coincide_ruta(ruta_definida, transicion):

    # dividir ambas rutas en segmentos
    partes_ruta = dividir_ruta(ruta_definida)
    partes_transicion = dividir_ruta(transicion)

    # si tienen distinta longitud, no coinciden
    if len(partes_ruta) != len(partes_transicion):
        return None

    # lista donde se almacenan los parametros capturados
    parametros = []

    # recorrer segmento a segmento
    for r, t in zip(partes_ruta, partes_transicion):

        # si el segmento de la ruta es un parametro (empieza con ":")
        if r.startswith(":"):
            parametros.append(t)  # guardar el valor correspondiente

        # si no es parametro, debe coincidir exactamente
        elif r != t:
            return None  # no coincide la ruta

    # si llega aqui, la ruta coincide
    return parametros
```

- `resolver_enrutamiento()`

```python
def resolver_enrutamiento():
    # obtiene la ruta absoluta del archivo actual
    base_dir = os.path.dirname(os.path.abspath(__file__))

    # construye rutas completas de entrada y salida
    input_path = os.path.join(base_dir, "input.txt")
    output_path = os.path.join(base_dir, "output.txt")

    # lista donde se almacenan las respuestas finales
    resultados = []

    #lectura de datos iniciales desde archivo
    with open(input_path, "r", encoding="utf-8") as archivo:

        # redefinimos input para leer linea a linea del archivo
        input = archivo.readline

        # leer numero de rutas
        N = int(input().strip())

        # lista donde se guardan las rutas
        # cada elemento es (ruta, contenido)
        rutas = []

        # leer las "N" rutas
        for _ in range(N):
            linea = input().strip()

            # separar en ruta y contenido
            ruta, contenido = linea.split(maxsplit=1)

            rutas.append((ruta, contenido))

        # leer numero de transiciones "M"
        M = int(input().strip())

        # procesamiento
        for _ in range(M):

            # leer una transicion
            transicion = input().strip()

            # bandera para saber si se encontro coincidencia
            encontrado = False

            # recorrer todas las rutas registradas
            for ruta, contenido in rutas:
                # verificar si coincide
                parametros = coincide_ruta(ruta, transicion)
                # si coincide la ruta
                if parametros is not None:

                    # inicializamos la salida con el contenido base
                    salida = contenido

                    # reemplazar placeholders {id} por valores reales
                    for valor in parametros:
                        salida = salida.replace("{id}", valor, 1)

                    # si NO hay {id}, pero hay parametros,
                    # se agregan al final (caso general)
                    if "{id}" not in contenido and parametros:
                        salida += " " + " ".join(parametros)

                    # guardar resultado
                    resultados.append(salida)

                    encontrado = True
                    break  # ya encontramos la ruta correcta

            # si no se encontro ninguna coincidencia
            if not encontrado:
                resultados.append("404 Not Found")

    # escritura de resultado en un archivo
    with open(output_path, "w", encoding="utf-8") as salida:

        # escribir cada linea en el archivo
        for linea in resultados:
            salida.write(linea + "\n")

    # impresion de resultado por terminal
    for linea in resultados:
        print(linea)
```

1.3. Lectura de Datos

El archivo de entrada contiene:

- Primera linea: un entero N (numero de rutas)
- N lineas: cada una contiene una ruta y su contenido
- Una linea: entero M (numero de transiciones)
- M lineas: rutas que representan transiciones del usuario

1.4. Representacion de Rutas

Cada ruta se representa como una cadena, por ejemplo:

- /
- /profile
- /user/:id

Para facilitar la comparacion, cada ruta se descompone en segmentos:

$$/\text{user}/:\text{id} \;\rightarrow\; [\text{user},\ :id]$$

$$/\text{user}/42 \;\rightarrow\; [\text{user},\ 42]$$

1.5. Estructura de Datos

Se utiliza una lista de pares:

$$(\text{ruta},\ \text{contenido})$$

Por ejemplo:

$$\text{rutas} = \{(/, \text{HomePage}), (/\text{profile}, \text{ProfilePage}), (/\text{user}/:\text{id}, \text{UserPage})\}$$

1.6. Procesamiento de Transiciones

Para cada transicion:

1. Se divide la ruta en segmentos
2. Se compara con cada ruta registrada
3. Se verifica coincidencia segmento por segmento

1.7. Regla de Coincidencia

Sean:

$$R = [r_1, r_2, \ldots, r_k], \quad T = [t_1, t_2, \ldots, t_k]$$

Entonces la ruta coincide si:

$$\forall i : \begin{cases} r_i = t_i, & \text{si no es parametro} \\ r_i =: x, & \text{(parametro, acepta cualquier valor)} \end{cases}$$

1.8. Captura de Parametros

Si un segmento es un parametro:

$$r_i =: id$$

entonces se captura:

$$\text{parametro} = t_i$$

y se añade al resultado:

$$\text{UserPage } + 42$$

1.9. Determinacion del Resultado

Para cada transicion:

- Si coincide con una ruta:

$$\text{salida} = \text{contenido}$$

- Si tiene parametros:

$$\text{salida} = \text{contenido} + \text{parametros}$$

- Si no coincide:

$$\text{salida} = \text{404 Not Found}$$

1.10. Estructuras de Datos Utilizadas

Estructura	**Tipo**	**Uso**
rutas	lista de pares	Almacenar rutas y contenido
segmentos	lista	Dividir rutas para comparacion
parametros	lista	Capturar valores dinamicos

1.11. Conclusion

El programa implementa un sistema de enrutamiento eficiente basado en comparacion de rutas segmentadas. Permite manejar rutas dinamicas con parametros y resolver transiciones en tiempo lineal respecto al numero de rutas y segmentos.

2. Cliente más fiel por socio

El Banco de la Nación tiene socios que recompensan a sus clientes con reembolsos como parte de promociones si realizan compras a través de sus terminales. Los socios valoran mucho a sus clientes y desean ofrecer mayores reembolsos a los más fieles. Debes ayudar a determinar al cliente más fiel para cada socio: aquel que realiza la mayor cantidad de compras a través de sus terminales. Escribe un programa que identifique al cliente más fiel de cada socio. Si hay varios clientes empatados, imprime el que tenga el ID más pequeño. Si no hay ninguno, imprime -1.

Formato de entrada

- La primera línea contiene tres números enteros N, M y S: el número de socios, el número de terminales y el número de transacciones, respectivamente, tales que:

$$1 \leq N, M, S \leq 1000$$

- Las siguientes M líneas contienen dos números (p, t), donde:

  - p es el ID del socio, con $1 \leq p \leq N$
  - t es el ID del terminal, con $1 \leq t \leq 10000$

  Los terminales son únicos para cada socio; es decir, ningún socio comparte el mismo ID de terminal.

- Las siguientes S líneas contienen dos números (c, t), donde:

  - c es el ID del cliente, con $1 \leq c \leq 100000$
  - t es el ID del terminal a través del cual realizó la compra, con $1 \leq t \leq 10000$

Formato de salida

Para cada socio, se debe mostrar una línea con dos valores:

```
socio_id   cliente_id
```

donde:

- `socio_id` es el identificador del socio
- `cliente_id` es el identificador del cliente más fiel

Los resultados deben estar ordenados por ID de socio en orden ascendente.

Si un socio no tiene transacciones, se debe imprimir:

```
socio_id   -1
```

Ejemplo de entrada

```
4 6 9
1 1
2 80
1 3
3 1000
2 150
4 20
1501 80
1502 150
1501 80
1501 150
1501 1
1501 3
1501 3
1502 1000
1503 1000
```

Ejemplo de Salida

```
1 1501
2 1501
3 1502
4 -1
```

Resolucion

En esta parte se tendra un archivo de entrada de la data, denominada input.txt. La salida del programa sera por terminal y un archivo de texto denominado output.tex, el cual estara en la misma ruta de donde se encuentra el archivo de entrada.

2.1. Objetivo del Programa

El objetivo del programa es identificar, para cada socio de un sistema, al cliente más fiel, definido como aquel que realiza la mayor cantidad de compras a través de los terminales asociados a dicho socio. En caso de empate entre varios clientes, se selecciona el cliente con el menor identificador (ID). Si un socio no registra transacciones, se devuelve el valor -1.

2.2. Estructura General del Programa

El programa se organiza en las siguientes etapas:

- Lectura y validación de datos de entrada
- Construcción de estructuras de datos
- Procesamiento de transacciones
- Determinación del cliente más fiel
- Escritura de resultados

2.3. Importación de Módulos

- `collections.defaultdict`: permite inicializar valores en cero automáticamente.
- `os`: permite manejar rutas de archivos de forma independiente del sistema operativo. Para nuestra solucion se usara el directorio actual donde se encuentra el programa $p2.py$ y el codigo de lectura sera:

```
#ruta actual
base_dir = os.path.dirname(os.path.abspath(__file__))

#ruta del archivo de entrada
input_path = os.path.join(base_dir, "input.txt")

#ruta del archivo de salida
output_path = os.path.join(base_dir, "output.txt")
```

2.4. Lectura de Datos

El archivo de entrada contiene:

- Primera línea: tres enteros N, M, S. La lectura de esta primera linea se hara como una cadena de string, luego validaremos si no esta vacia.

```python
#apertura de archivo de entrada
with open(input_path, "r", encoding="utf-8") as archivo:

    # se redefine input para leer linea a linea del archivo
    input = archivo.readline

    # VALIDAR PRIMERA LINEA
    # lectura de la primera linea del archivo y limpia de la
    # cadena(elimina espacios y cadenas de terminacion final)
    linea = input().strip()

    #si la linea esta vacia, se detiene el programa
    if not linea:
        raise ValueError("Error: la primera linea esta vacia")
```

Luego se valida que contegan tres valores y que estos sean enteros.

```python
# convierte una cadena en una lista separados por espacios  "1 2 3" -> ["1" "2" "3"]
partes = linea.split()

# deben ser exactamente 3 valores
if len(partes) != 3:
    raise ValueError("Error: la primera linea debe contener exactamente 3 enteros")

# N - numero de socios
# M - numero de terminales
# S - numero de transacciones
try:
    # ["1" "2" "3"] -> [1 2 3]
    N, M, S = map(int, partes)
except:
    raise ValueError("Error: los valores deben ser enteros")

# validacion de rango de N, M y S
if not (1 <= N <= 1000):
    raise ValueError(f"Error: N fuera de rango (N={N})")
```

La ultima validacion es que se cumpla:

$$1 \leq N, M, S \leq 1000$$

```python
# validacion de rango de N, M y S
if not (1 <= N <= 1000):
    raise ValueError(f"Error: N fuera de rango (N={N})")

if not (1 <= M <= 1000):
    raise ValueError(f"Error: M fuera de rango (M={M})")

if not (1 <= S <= 1000):
    raise ValueError(f"Error: S fuera de rango (S={S})")
```

- M líneas: pares (p, t) que representan socio y terminal
- S líneas: pares (c, t) que representan cliente y terminal

2.5. Mapeo de Terminales

Se construye un diccionario:

$$t \mapsto p$$

donde:

- t: identificador del terminal
- p: identificador del socio

Esto permite identificar rápidamente a qué socio pertenece cada terminal.

```python
# mapeo terminal por socio, se usara diccionario { t:p }
# clave: terminal(t)
# valor: socio(p)
terminal_to_socio = {}
# lee las M lineas de terminales
for _ in range(M):
    p, t = map(int, input().split())

    # se guarda la relacion
    terminal_to_socio[t] = p
```

2.6. Estructura de Conteo

Se define la estructura:

$$\text{conteo}[p][c] = \text{número de compras del cliente } c \text{ en el socio } p$$

Utilizando:

- Diccionario principal indexado por socio
- `defaultdict(int)` para conteo automático

Esto es, se crea diccionario de diccionario, donde la clave del diccionario principal sera un socio c, el valor del diccionario principal sera un diccionario que contenga como clave cliente y como valor cantidad.

$$\text{conteo} = \{\text{socio} \mapsto \{\text{cliente} \mapsto \text{cantidad}\}\}$$

Inicialmente el diccionario no contiene valor por cada socio, pero existe el socio que esta bien definido, esto se resuelve usando $defaultdict(int)$, el cual deja en 0 automaticamente.

```
# definicion de la estructura de conteo
# conteo[p][c] = número de compras del cliente c en el socio p
# defaultdict(int) inicializa automáticamente en 0
conteo = {i: defaultdict(int) for i in range(1, N + 1)}
```

Por ejemplo inicialmente en la primera vez que se ejecuta el codigo para $N = 3$, este tendra la siguiente estructura:

$$\text{conteo} = \left\{ \begin{array}{l} 1: \ \text{defaultdict(int)} \\ 2: \ \text{defaultdict(int)} \\ 3: \ \text{defaultdict(int)} \end{array} \right\}$$

2.7. Procesamiento de Transacciones

Para cada transacción (c, t):

1. Se verifica si el terminal existe
2. Se obtiene el socio correspondiente p a través del diccionario `terminal_to_socio`.
3. Se incrementa el contador, a travez del diccionario de diccionarios, en su respectivo socio:

$$\text{conteo}[p][c] \leftarrow \text{conteo}[p][c] + 1$$

```
#procesamiento de transacciones,se lee S transacciones
for _ in range(S):
    c, t = map(int, input().split())

    #verificar si el terminal pertenece a algun socio
    if t not in terminal_to_socio:
        continue  # ignorar si no existe

    #obtener el socio correspondiente
    socio = terminal_to_socio[t]

    #incrementar el contador
    conteo[socio][c] += 1
```

2.8. Determinación del Cliente Más Fiel

En esta parte se barre un todos los socios a traves de un for, asi para cada socio en el diccionario de diccionario, se busca con la clave "*socio*" y nos entregran el valor un diccionario el cual lo llamaremos "*clientes*". Este nuevo diccionario su clave seran cliente "c" y clave el numero de compras hechos por ese cliente "*compras*". Si no hay transaciones se pone el resultado al socio por -1. Luego se inicializa las variables auxiliares "`mejor_cliente` $= -1$" y "`max_compras` $= -1$". Asi para cada "*cliente*", este lee "c" y "*compras*" y este compara con las compras maximas y va seleccionando el que tiene el maximo de compras.

Para cada socio p:

- Si no tiene transacciones:

$$\text{resultado} = -1$$

- En caso contrario, se busca:

$$\max_{c} \ \text{conteo}[p][c]$$

- En caso de empate:

$$\text{mín}(\text{ID del cliente})$$

```python
#determinacion de cliente mas fiel por socio
resultado = [] #inicializamos
for socio in range(1, N + 1):
    clientes = conteo[socio]

    # caso: no hay transacciones
    if not clientes:
        resultado.append(f"{socio} -1")
        continue

    #variables auxiliares para inicializacion
    mejor_cliente = -1
    max_compras = -1

    #busqueda del maximo
    for c, compras in clientes.items():
        # 1. Mayor numero de compras
        # 2. Si hay empate, usar menor ID
        if (compras > max_compras or
            (compras == max_compras and c < mejor_cliente)):

            max_compras = compras
            mejor_cliente = c

    resultado.append(f"{socio} {mejor_cliente}")
```

2.9. Escritura de Resultados

Los resultados se almacenan en el archivo de salida en el formato:

$$p \quad c$$

donde:

- p: socio
- c: cliente más fiel o -1

La salida por terminal sera:

```
PS D:\CICLO 2026-1 UNI\Desarrollo-de-Software---Tareas> & C:/Users/Usuario/AppData
.exe "d:/CICLO 2026-1 UNI/Desarrollo-de-Software---Tareas/Tarea 1/P2/p2.py"
1 1501
2 1501
3 1502
4 -1
```

La salida por archivo sera:

```
output.txt M
1   1 1501
2   2 1501
3   3 1502
4   4 -1
```

2.10. Estructuras de Datos Utilizadas

Estructura	Tipo	Uso
terminal_to_socio	diccionario	Mapear terminal a socio
conteo	diccionario anidado	Contar compras por cliente
resultado	lista	Almacenar salida final

2.11. Conclusión

El programa implementa un algoritmo eficiente basado en estructuras de datos tipo diccionario para realizar conteo acumulativo.